

Object Oriented Programming

Prof. Dr. Muhammad Usman
Department of Computer Science
UET Mardan

Email: usman@uetmardan.edu.pk

SEPARATING INTERFACE FROM IMPLEMENTATION

Interfaces

- Interfaces define and standardize the ways in which things, such as people and systems, interact with one another
- The interface of a class describes **what services** clients of a class can use and **how to request** those services, but **not how the class carries out** these services
- A class's public interface consists of the class's public member functions

Separating the Interface from the Implementation

- So far, we wrote **class definition** and its **member function implementation** in **one file**
- However, it's better software engineering practice to define member functions outside the class definition, so that their implementation details can be hidden from the client.

SEPARATING INTERFACE FROM IMPLEMENTATION

- In this way the implementation will become separate from interface
- Changes in the implementation will not affect the clients as long as the public interface remains constant

Defining a Class's Interface with Function Prototypes

- The interface contains the **class definition** having data members and prototypes of member functions
- This *interface* will be saved as header file with **extension .h**
- This file tells the compiler about the functions name, its return types, the types of function parameters and class private data members because compiler must know the data members of the class to determine how much memory to reserve for each object of the class

SEPARATING INTERFACE FROM IMPLEMENTATION

Defining Member Functions in a Separate Source-Code File

- The definitions of the functions declared in **.h** file will be written in a **separate .cpp file**
- This file is the actual implementation file
- Changes in this file will not effect **.h** file as long as the private data members and function declaration remains the same
- Each member function name in the function headers in this file is preceded by the class name and **::** (binary scope resolution operator), otherwise the compiler will consider it a global function
- We must include the **.h** file in this **.cpp** file to indicate that these implementations belong to the class in **.h** file
- When compiling **.cpp** file the compiler uses the information in **.h** file to ensure that the **.cpp** file must match the prototypes in **.h** file and each member function knows about the class's data members and other member functions

SEPARATING INTERFACE FROM IMPLEMENTATION

Using the .h interface file

- To use the interface file we must include the **.h** file in our program
- Separating implementation from interface does not affect the way we write programs
- It affects only how the program is compiled and linked

The Compilation and Linking Process

- Often a class's interface and implementation are created and compiled by one programmer and used by a separate programmer who implements the client code that uses the class
- When the client code is compiled, the compiler uses the class definition in **.h** to ensure that the main function create and manipulate objects of class in **.h** file correctly

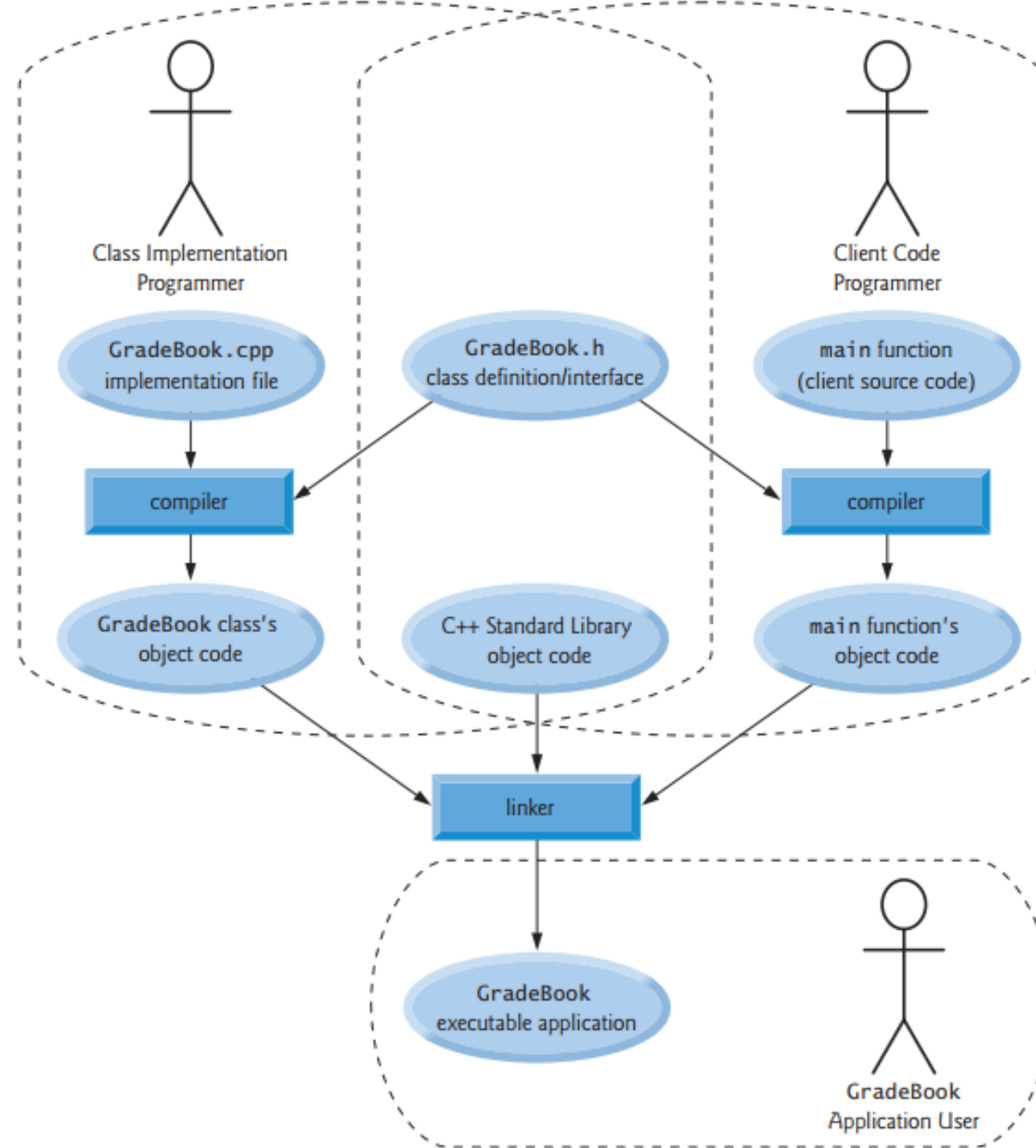
SEPARATING INTERFACE FROM IMPLEMENTATION

Header files

- Class definitions and function prototypes
- Included in each file using `class`
 - `#include`
- File extension `.h`

Source-code files

- Member function definitions
- Same base name
 - Convention
- Compiled and linked



SEPARATING INTERFACE FROM IMPLEMENTATION

Advantage

- Easier to modify programs

Disadvantage

- Header files contain
 - Portions of implementation
 - Inline member functions
 - Hints about other implementation
 - Private members

We Can hide more with **proxy** class

```

1.  //time1.h
2  // Declaration of class Time.
3  // Member functions are defined in time1.cpp
5  // prevent multiple inclusions of header file
6  #ifndef TIME1_H
7  #define TIME1_H
9  // Time abstract data type definition
10 class Time {
12 public:
13     Time(); // constructor
14     void setTime( int, int, int ); // set time
15     void printUniversal(); // print universal-time format
16     void printStandard(); // print standard-time format
18 private:
19     int hour; // 0 - 23 (24-hour clock format)
20     int minute; // 0 - 59
21     int second; // 0 - 59
22
23 }; // end class Time
24
25 #endif

```

Preprocessor code to prevent multiple inclusions.

Code between these directives

Preprocessor directive defines name **TIME1_H**.

"If not defined"

Naming convention: header file name with underscore replacing period.

EXAMPLE

#ifndef stands for "**if not defined**". It is a **preprocessor directive** used to prevent a header file from being included **more than once** in the same compilation — this is called an **include guard**.

```
1  // time1.cpp
2  // Member-function definitions for class Time.
3  #include <iostream>
4
5  using std::cout;
6
7  #include <iomanip>
8
9  using std::setfill;
10 using std::setw;
11
12 // include definition of class Time from time1.h
13 #include "time1.h"
14
15 // Time constructor initializes
16 // Ensures all Time objects start at 0
17 Time::Time()
18 {
19     hour = minute = second = 0;
20
21 } // end Time constructor
22
```

Include header file
time1.h.

Name of header file enclosed
in quotes; angle brackets
cause preprocessor to assume
header part of C++ Standard
Library.

```
23 // Set new Time value using universal time. Perform validity
24 // checks on the data values. Set invalid values to zero.
25 void Time::setTime( int h, int m, int s )
26 {
27     hour = ( h >= 0 && h < 24 ) ? h : 0;
28     minute = ( m >= 0 && m < 60 ) ? m : 0;
29     second = ( s >= 0 && s < 60 ) ? s : 0;
30
31 } // end function setTime
32
33 // print Time in universal format
34 void Time::printUniversal()
35 {
36     cout << setfill( '0' ) << setw( 2 ) << hour << ":"
37         << setw( 2 ) << minute << ":"
38         << setw( 2 ) << second;
39
40 } // end function printUniversal
41
```

```
42 // print Time in standard format
43 void Time::printStandard()
44 {
45     cout << ( ( hour == 0 || hour == 12 ) ? 12 : hour % 12 )
46         << ":" << setfill( '0' ) << setw( 2 ) << minute
47         << ":" << setw( 2 ) << second
48         << ( hour < 12 ? " AM" : " PM" );
49
50 } // end function printStandard
```

MAINPROG.CPP (1 OF 2)

```
1 // mainprog.cpp
2 // Program to test class Time.
3 // NOTE: This file must be compiled with time1.cpp.
4 #include <iostream>
5
6 using std::cout;
7 using std::endl;
8
9 // include definition of class Time from time1.h
10 #include "time1.h"
11
12 int main()
13 {
14     Time t; // instantiate object t of class Time
15
16     // output Time object t's initial values
17     cout << "The initial universal time is ";
18     t.printUniversal(); // 00:00:00
19     cout << "\nThe initial standard time is ";
20     t.printStandard(); // 12:00:00 AM
21
22     t.setTime( 13, 27, 6 ); // change time
23
```

Include header file **time1.h** to ensure correct creation/manipulation and determine size of **Time** class object.

```
24 // output Time object t's new values
25 cout << "\n\nUniversal time after setTime is ";
26 t.printUniversal(); // 13:27:06
27 cout << "\nStandard time after setTime is ";
28 t.printStandard(); // 1:27:06 PM
29
30 t.setTime( 99, 99, 99 ); // attempt invalid settings
31
32 // output t's values after specifying invalid values
33 cout << "\n\nAfter attempting invalid settings:"
34     << "\nUniversal time: ";
35 t.printUniversal(); // 00:00:00
36 cout << "\nStandard time: ";
37 t.printStandard(); // 12:00:00 AM
38 cout << endl;
39
40 return 0;
41
42 } // end main
```

```
The initial universal time is 00:00:00
The initial standard time is 12:00:00 AM

Universal time after setTime is 13:27:06
Standard time after setTime is 1:27:06 PM
```

```
#ifndef FILENAME_H
```

```
#define FILENAME_H
```

```
// your header file content here
```

```
#endif
```

If you include the same header file multiple **times** (directly or indirectly), the compiler will see duplicate declarations and **throw** an error. `#ifndef` prevents **this**.

`#ifndef` stands for **"if not defined"**.

It is a **preprocessor directive** used to prevent a header file from being included **more than once** in the same compilation — this is called an **include guard**.

```
**Without guard – Problem:**  
...
```

```
mainprog.cpp
```

```
└─ includes interface.h    ✓ first time – OK
```

```
└─ includes interfdef.cpp
```

```
    └─ includes interface.h again ✗ DUPLICATE – ERROR
```

```
...
```

```
**With guard – Safe:**
```

```
...
```

```
mainprog.cpp
```

```
└─ includes interface.h    ✓ first time – defined
```

```
└─ includes interfdef.cpp
```

```
    └─ includes interface.h again – skipped ✓
```

CONST (CONSTANT) OBJECTS AND CONST MEMBER FUNCTIONS

- Some objects need to be modifiable and some do not
- Principle of least privilege
 - Only allow modification of necessary objects
- The programmer may use the keyword `const` to specify that an object is not modifiable
 - Any attempt to modify the Const object is a **syntax error**

▪ Example

```
const Time noon( 12, 0, 0 );
```

- Declares const object noon of class Time and Initializes to 12

CONST (CONSTANT) OBJECTS AND CONST MEMBER FUNCTIONS

- Member functions for **const** objects must also be **const**
 - Cannot modify object
 - Member functions declared **const** can not modify the object, the compiler disallows this
- Specify **const** in both prototype and definition of member function
 - Prototype
 - Const After parameter list
 - Definition
 - Const Before beginning left brace(i.e., after the function parameter list)

const Member Function

- ✗ A **const** member function guarantees that it will never modify any of its class's member data.

```
//constfu.cpp demonstrates const member functions
class aClass
{
private:
    int alpha;
public:
    void nonFunc()                // non-const member function
    {
        alpha = 99;              // OK
    }

    void conFunc() const          // const member function
    {
        alpha = 99;              // ERROR: can't modify a member
    }
};
```

Distance Class and Use of **const**

(1/3)

```
// const member functions & const arguments to member functions
#include <iostream>
using namespace std;

class Distance                                // English Distance class
{
private:
    int feet;
    float inches;
public:
    Distance() : feet(0), inches(0.0)         // constructor (no args)
    { }                                       // constructor (two args)

    Distance(int ft, float in) : feet(ft), inches(in)
    { }

    void getdist(){                          // get length from user
        cout << "\nEnter feet: "; cin >> feet;
        cout << "Enter inches: "; cin >> inches;
    }
}
```

Distance Class and Use of `const`

(2/3)

```
void showdist() const           // display distance
{ cout << feet << "\'-" << inches << '\\"'; }
Distance add_dist(const Distance&) const; // add
};

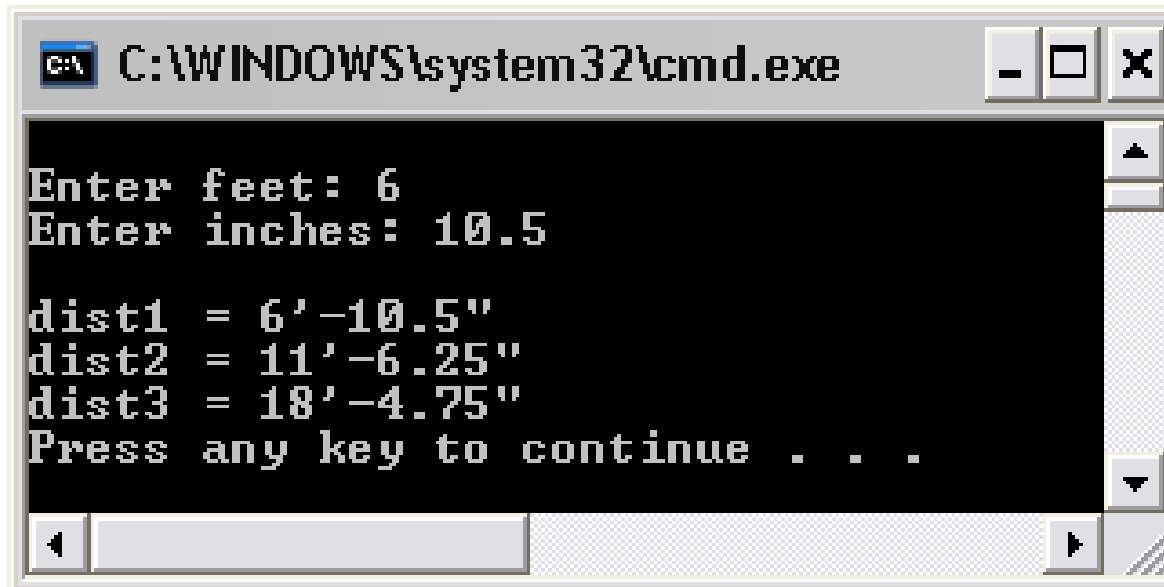
//add this distance to d2, return the sum
Distance Distance::add_dist(const Distance& d2) const
{
    Distance temp;                // temporary variable
    // feet = 0;                  // ERROR: can't modify this
    // d2.feet = 0;               // ERROR: can't modify d2
    temp.inches = inches + d2.inches; // add the inches
    if(temp.inches >= 12.0)          // if total exceeds 12.0,
    { temp.inches -= 12.0;           // then decrease inches by 12.0
      temp.feet = 1;                 // and increase feet by 1
    }
    temp.feet += feet + d2.feet;    // add the feet
    return temp;                   // return temp object
}
```

Distance Class and Use of `const`

(3/3)

```
int main()
{
    Distance dist1, dist3;           // define two lengths
    Distance dist2(11, 6.25);        // define, initialize dist2
    dist1.getdist();                 // get dist1 from user
    dist3 = dist1.add_dist(dist2);    // dist3 = dist1 + dist2
    //display all lengths
    cout << "\ndist1 = "; dist1.showdist();
    cout << "\ndist2 = "; dist2.showdist();
    cout << "\ndist3 = "; dist3.showdist();
    cout << endl;

    return 0;
}
```



```
C:\WINDOWS\system32\cmd.exe

Enter feet: 6
Enter inches: 10.5

dist1 = 6'-10.5"
dist2 = 11'-6.25"
dist3 = 18'-4.75"
Press any key to continue . . .
```

const Member Function Arguments

- ✖ if an argument is passed to an **ordinary function** by reference, and **you don't want the function to modify it**, the argument should be made `const` in the function declaration (and definition). This is true of member functions as well.
`Distance Distance::add_dist(const Distance& d2) const{`
- ✖ In above line, argument to `add_dist()` is passed by reference, and we want to make sure that it won't modify this variable, which is `dist2` in `main()`.
- ✖ Therefore we make the argument `d2` to `add_dist()` `const` in both declaration and definition.

const Objects

- ✖ As we can apply `const` to variables of basic types such as `int` to keep them from being modified.
- ✖ In a similar way, we can apply `const` to `objects` of `classes`. When an `object` is declared as `const`, `you can't modify it`.
- ✖ It follows that `you can use only const member functions with it`, because they're the only ones that guarantee not to modify it. e.g. A football field (for American-style football) is exactly 300 feet long. If we were to use the length of a football field in a program, it would make sense to make it `const`, because changing it would represent the end of the world for football fans.

const Objects

(1/2)

```
// constObj.cpp constant Distance objects
#include <iostream>
using namespace std;
class Distance                                // English Distance class
{
private:
    int feet;
    float inches;
public:
    // 2-arg constructor
    Distance(int ft, float in) : feet(ft), inches(in)
    { }

    void getdist()                            // user input; non-const function
    {
        cout << "\nEnter feet:" ; cin >> feet;
        cout << "Enter inches:" ; cin >> inches;
    }
}
```

const Objects

(2/2)

```
void showdist() const           // display distance; const function
{
    cout << feet << "\'-" << inches << '\\"';
}
};

int main()
{
    const Distance football(300, 0);

    // football.getdist();    // ERROR: getdist() not const

    cout << "football = " ;
    football.showdist();      // OK
    cout << endl;
    return 0;
}
```

```
1 // time5.h
2 // Definition of class Time.
3 // Member functions defined in time5.cpp.
4 #ifndef TIME5_H
5 #define TIME5_H
6
7 class Time {
8
9 public:
10     Time( int = 0, int = 0, int = 0 ); // default constructor
11
12     // set functions
13     void setTime( int, int, int ); // set time
14     void setHour( int );           // set hour
15     void setMinute( int );         // set minute
16     void setSecond( int );         // set second
17
18     // get functions (normally declared const)
19     int getHour() const;           // return hour
20     int getMinute() const;         // return minute
21     int getSecond() const;         // return second
```

ANOTHER EXAMPLE

Declare **const** get functions.

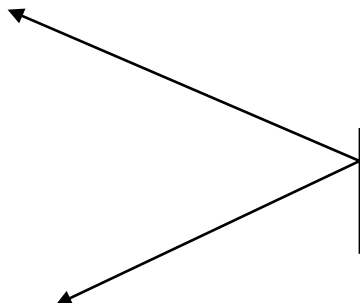
```
22
23 // print functions (normally declared const)
24 void printUniversal() const; // print universal time
25 void printStandard(); // print standard time
26
27 private:
28     int hour; // 0 - 23 (24-hour clock format)
29     int minute; // 0 - 59
30     int second; // 0 - 59
31
32 }; // end class Time
33
34 #endif
```

Declare **const** function
printUniversal.

```
1  // time5.cpp
2  // Member-function definitions for class Time.
3  #include <iostream>
4
5  using std::cout;
6
7  #include <iomanip>
8
9  using std::setfill;
10 using std::setw;
11
12 // include definition of class Time from time5.h
13 #include "time5.h"
14
15 // constructor function to initialize private data;
16 // calls member function setTime to set variables;
17 // default values are 0 (see class definition)
18 Time::Time( int hour, int minute, int second )
19 {
20     setTime( hour, minute, second );
21
22 } // end Time constructor
```

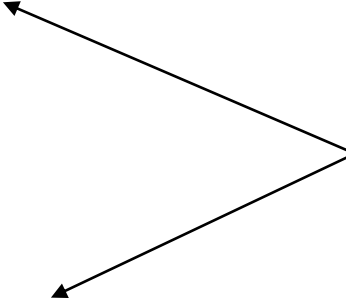
```
24 // set hour, minute and second values
25 void Time::setTime( int hour, int minute, int second )
26 {
27     setHour( hour );
28     setMinute( minute );
29     setSecond( second );
30
31 } // end function setTime
32
33 // set hour value
34 void Time::setHour( int h )
35 {
36     hour = ( h >= 0 && h < 24 ) ? h : 0;
37
38 } // end function setHour
39
40 // set minute value
41 void Time::setMinute( int m )
42 {
43     minute = ( m >= 0 && m < 60 ) ? m : 0;
44
45 } // end function setMinute
```

```
47 // set second value
48 void Time::setSecond( int s )
49 {
50     second = ( s >= 0 && s < 60 ) ? s : 0;
51
52 } // end function setSecond
53
54 // return hour value
55 int Time::getHour() const
56 {
57     return hour;
58
59 } // end function getHour
60
61 // return minute value
62 int Time::getMinute() const
63 {
64     return minute;
65
66 } // end function getMinute
67
```



const functions do not
modify objects.

```
68 // return second value
69 int Time::getSecond() const
70 {
71     return second;
72 }
73 // end function getSecond
74
75 // print Time in universal format
76 void Time::printUniversal() const
77 {
78     cout << setfill( '0' ) << setw( 2 ) << hour << ":"
79         << setw( 2 ) << minute << ":"
80         << setw( 2 ) << second;
81 }
82 // end function printUniversal
83
84 // print Time in standard format
85 void Time::printStandard() // note lack of const declaration
86 {
87     cout << ( ( hour == 0 || hour == 12 ) ? 12 : hour % 12 )
88         << ":" << setfill( '0' ) << setw( 2 ) << minute
89         << ":" << setw( 2 ) << second
90         << ( hour < 12 ? " AM" : " PM" );
91 }
92 // end function printStandard
```



const functions do not
modify objects.

```
1 // Fig. 10.3: fig10_03.cpp
2 // Attempting to access a const object with non-const member functions.
3 #include "Time.h" // include Time class definition
4
5 int main()
6 {
7     Time wakeUp( 6, 45, 0 ); // non-constant object
8     const Time noon( 12, 0, 0 ); // constant object
9
10    // OBJECT MEMBER FUNCTION
11    wakeUp.setHour( 18 ); // non-const non-const
12
13    noon.setHour( 12 ); // const non-const
14
15    wakeUp.getHour(); // non-const const
16
17    noon.getMinute(); // const const
18    noon.printUniversal(); // const const
19
20    noon.printStandard(); // const non-const
21 } //endmain
```

Note that non-**const** constructor can initialize **const** object.

Declare **noon** a **const** object.

CONST (CONSTANT) OBJECTS AND CONST MEMBER FUNCTIONS

const with Constructor and Destructor

- Constructor and destructors can not be const
- A constructor must be allowed to modify an object so that the object can be initialized properly
- A destructor must be able to perform its termination housekeeping before an object is destroyed
- A constructor must be a non-const member function, it can still be called for a const object

CONST VS NON-CONST MEMBER FUNCTIONS

Member initializer Syntax

- Initializing with member initializer syntax
 - can be used for all data members
 - must be used for
 - `const` data members
 - Data members that are references

```
// constructor
Time::Time(int h, int m, int s)
    :hour( h ),    // initializer for non-const member
    minute( m ),  // required initializer for const member
    second( s )   // required initializer for const member
{ }
```

```
1  // membinit.cpp
2  // Using a member initializer to initialize a
3  // constant of a built-in data type.
4  #include <iostream>
5
6  using std::cout;
7  using std::endl;
8
9  class Increment {
10
11  public:
12      Increment( int c = 0, int i = 1 ); // default constructor
13
14      void addIncrement()
15      {
16          count += increment;
17
18      } // end function addIncrement
19
20      void print() const; // prints count and increment
21
```

```

22 private:
23     int count;
24     const int increment; // const data member
25
26 }; // end class Increment
27
28 // constructor
29 Increment::Increment( int c, int i )
30 : count( c ), increment( i )
31 {
32     // empty body
33 }
34 // end Increment constructor
35
36 // print count and increment
37 void Increment::print() const
38 {
39     cout << "count = " << count
40         << ", increment = " << increment << endl;
41
42 } // end function print

```

Declare increment as **const** data member.

Member initializer list separated from parameter list by colon.

Member initializer syntax can be used for non-**const** data member **count**.

Member initializer syntax must be used for **const** data member **increment**.

Member initializer consists of data member name (**increment**) followed by parentheses containing initial value (**c**).

```
45  int main()
46  {
47      Increment value( 10, 5 );
48
49      cout << "Before incrementing: ";
50      value.print();
51
52      for ( int i = 0; i < 3; i++ ) {
53          value.addIncrement();
54          cout << "After increment " << i + 1 << ": ";
55          value.print();
56      }
57
58      return 0;
59
60  } // end main
```

```
Before incrementing: count = 10, increment = 5
After increment 1: count = 15, increment = 5
After increment 2: count = 20, increment = 5
After increment 3: count = 25, increment = 5
```

DEFAULT MEMBERWISE ASSIGNMENT

Assigning Objects

- The Assignment operator (=) can be used to assign an object to another object of the **same type**
- Such assignment is by default performed by member wise copy
 - Each member of one object is copied (assigned) individually to the same member in another object
- Memberwise copy can cause serious problems when used with a class whose data members contain dynamically allocated storage
 - **shallow copy**

```

class MyString
{ private:
char* str; // dynamically allocated
int len;
public:
MyString(const char* s)
{ len = strlen(s);
str = new char[len + 1]; // allocate memory
strcpy(str, s); }
~MyString()
{ delete[] str; // free memory }
};

int main() {
MyString s1("Hello");
MyString s2 = s1; // default copy constructor (shallow copy)
}

.....

```

Three Serious Problems

1. **✗ Double Deletion** (Dangling Pointer) ``
 - s1 destroyed → deletes memory at address X
 - s2 destroyed → tries to **delete** same address X again
 - CRASH / undefined behavior
2. **✗ Unintended Modification**
 - s2.str[0] = 'Y'; // meant to change only s2
 - // but s1 is ALSO changed! (same memory)
3. **✗ Memory Leak / Corruption**
 - s2 = s1; // s2's original memory is never freed
 - // → memory leak

What Happens in Memory:

```

...
s1.str ┌───┐
        │   │
s2.str ┌───┐ ──→ [ H | e | l | l | o | \0 ]
...

```

DEFAULT MEMBERWISE ASSIGNMENT

Passing, returning objects

- Objects may be passed as function arguments
- Object may be returned from functions
- Such passing and returning is performed by **call-by-value** by default
- A copy of the object is passed or returned
- It uses copy constructor, which copies original object's values into new object

```
Wall(Wall &obj) {  
    length = obj.length;  
    height = obj.height;  
}
```

STATIC CLASS MEMBERS

- Each object of a class has its **own copy** of the data members of the class
- In certain cases one copy of a variable should be shared by all objects of a class
- A **static** class variable is used for these and other reasons
- A **static** class variable represents “**class-wide**” information
- Efficient when single copy of data is enough
- In other words, **static** variables are the **property of the class** and not of a specific object
- The declaration of static member begins with keyword **static**
- **static** members look like global but these have class scope
- **static** members can be public, private or protected

STATIC CLASS MEMBERS

- **static** members must be initialized once at file scope, because they are independent of the number of objects and hence must be initialized independently
- A class **static** members are only accessible to objects of same class or they can be accessed without any object of the class
- **static** keyword appears **only in the declaration** inside the class.
- The **definition outside the class** uses **ClassName::memberName** with **no static**.

STATIC CLASS MEMBERS

- A class public `static` members can be accessed through the class name using scope resolution operator

```
Student::count;
```

- A class private and protected `static` members must be accessed through public member functions or `friend function of the class`
- To access `private` or `protected static` members when no object of the class exist, a `public static` member function must be provided preceded with binary scope resolution operator and the class name as,

```
Student::getCount();
```

- `static` member function can not use non-static data, functions and `this` pointer because `static` members are independent of the objects of the class

Static Class Data

- ✖ If a data item in a class is declared as *static*, only one such item is created for the entire *class*, no matter how many *objects* there are.
- ✖ A *static* data item is useful when all objects of the same class must *share* a common item of information.
- ✖ A *member variable defined* as *static* has characteristics similar to a normal static variable: It is *visible* only *within* the *class*, but its *lifetime* is the *entire program*. It continues to exist even if there are no *objects* of the *class*.
- ✖ a normal static variable is used to retain information between calls to a function, *static class member data* is used to *share information among the objects of a class*.

Static Class Data

- ✖ Why would you want to use *static* member data? As an example, suppose an object needed to know *how many other objects of its class* were in the program.
- ✖ In a road-racing game, for example, a race car might want to know how many other cars are still in the race.
- ✖ In this case a *static* variable *Total_Cars* could be included as a member of the class. All the objects would have access to this variable. It would be the same variable for all of them; they would all see the same number of *Total_Cars*.

STATIC DATA CLASS

(1/2)

```
// statdata.cpp demonstrates a simple static data member
#include <iostream>
#include <stdlib.h>
using namespace std;
class Car{
private:
    static int Total_Cars; // only one data item for all objects
                           // note: "declaration" only!
public:
    Car()                  // increments count when object created
    { Total_Cars++; }

    int How_Many()         // returns count
    { return Total_Cars; }

    ~Car()
    { Total_Cars--; }

};
int Car::Total_Cars = 0;   // *definition* of count
```

STATIC DATA CLASS

(2/2)

```
int main()
{
    Car Toyota, Honda, Suzuki; // create three objects
    cout << Toyota.How_Many() << " Cars are in Race" << endl;
    cout << Honda.How_Many() << " Cars are in Race" << endl;
    // each object sees the same data
    cout << Suzuki.How_Many() << " Cars are in Race" << endl;

    Car *Pajero = new Car;

    cout << Suzuki.How_Many() << " Cars are in Race" << endl;
    cout << Pajero->How_Many() << " Cars are in Race" << endl;

    delete Pajero;

    cout << Honda.How_Many() << " Cars are in Race" << endl;

    return 0;
}
```

Static Class Data

- ✖ *Ordinary variables* are usually declared (the compiler is told about their name and type) and defined (the compiler sets aside memory to hold the variable) in the **same statement**. e.g. `int a;`
- ✖ *Static member data*, on the other hand, requires two separate statements.
 - + The variable's *declaration* appears in the *class definition*,
 - + but the variable is actually *defined outside* the *class*, in much the same way as a global variable.
- ✖ If *static* member data were defined inside the class, it would violate the idea that a class definition is only a blueprint and does not set aside any memory.

```
1  // employee2.h
2  // Employee class definition.
3  #ifndef EMPLOYEE2_H
4  #define EMPLOYEE2_H
5
6  class Employee {
7
8  public:
9      Employee( const char *, const char * ); // constructor
10     ~Employee(); // destructor
11     const char *getFirstName() const; // return first
12     const char *getLastName() const; // return last
13
14     // static member function
15     static int getCount(); // return # objects instantiated
16
17 private:
18     char *firstName;
19     char *lastName;
20
21     // static data member
22     static int count; // number of objects instantiated
23
24 }; // end class Employee
25 #endif
```

static member function
can only access **static** data
members and member
functions.

static data member is
class-wide data.

```
1  // employee2.cpp
2  // Member-function definitions for class Employee.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  #include <new>          // C++ standard new operator
9  #include <cstring>      // strcpy and strlen prototypes
10
11 #include "employee2.h" // Employee class
12
13 // define and initialize static data member
14 int Employee::count = 0;
15
16 // define static member function that
17 // Employee objects instantiated
18 int Employee::getCount()
19 {
20     return count;
21
22 } // end static function getCount
```

Initialize **static** data member exactly once at file scope.

static member function accesses **static** data member **count**.

```
23
24 // constructor dynamically allocates space for
25 // first and last name and uses strcpy to copy
26 // first and last names into the object
27 Employee::Employee( const char *first, const char *last )
28 {
29     firstName = new char[ strlen( first ) + 1 ];
30     strcpy( firstName, first );
31
32     lastName = new char[ strlen( last ) + 1 ];
33     strcpy( lastName, last );
34
35     ++count; // increment static count of employees
36
37     cout << "Employee constructor for " << firstName
38         << ' ' << lastName << " called." << endl;
39
40 } // end Employee constructor
41
42 // destructor deallocates dynamically allocated memory
43 Employee::~Employee()
44 {
45     cout << "~Employee() called for " << firstName
46         << ' ' << lastName << endl;
47
```

new operator dynamically allocates space.

Use **static** data member to store total **count** of employees.

```

48  delete [] firstName; // recapture memory
49  delete [] lastName; // recapture memory
50
51  --count; // decrement static count of employees
52
53 } // end destructor ~Employee
54
55 // return first name of employee
56 const char *Employee::getFirstName() const
57 {
58     // const before return type prevents client from modifying
59     // private data; client should copy returned string before
60     // destructor deletes storage to prevent undefined pointer
61     return firstName;
62
63 } // end function getFirstName
64
65 // return last name of employee
66 const char *Employee::getLastName() const
67 {
68     // const before return type prevents client from modifying
69     // private data; client should copy returned string before
70     // destructor deletes storage to prevent undefined pointer
71     return lastName;
72
73 } // end function getLastName

```

Operator **delete** deallocates

Use **static** data member to store total **count** of employees.

```
1  // mainprog.cpp
2  // Driver to test class Employee.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  #include <new>          // C++ standard new operator
9
10 #include "employee2.h" // Employee class definition
11
12 int main()
13 {
14     cout << "Number of employees before instantiation is "
15         << Employee::getCount() << endl; // use class name
16
17     Employee *e1Ptr = new Employee( "Susan", "Baker" );
18     Employee *e2Ptr = new Employee( "Susan", "Baker" );
19
20     cout << "Number of employees after instantiation is "
21         << e1Ptr->getCount();
22 }
```

new operator dynamically
allocates space.

static member function
can be invoked on any object
of class.

```
23  cout << "\n\nEmployee 1: "
24      << e1Ptr->getFirstName()
25      << " " << e1Ptr->getLastName()
26      << "\n\nEmployee 2: "
27      << e2Ptr->getFirstName()
28      << " " << e2Ptr->getLastName() << "\n\n";
29
30  delete e1Ptr; // recapture memory
31  e1Ptr = 0;    // disconnect pointer from free-store space
32  delete e2Ptr; // recapture memory
33  e2Ptr = 0;    // disconnect pointer from free-store space
34
35  cout << "Number of employees after"
36      << Employee::getCount() << endl;
37
38  return 0;
39
40 } // end main
```

Operator **delete** deallocates
static member function
invoked using binary scope
resolution operator (no
existing class objects).

OUTPUT

Number of employees before instantiation is 0

Employee constructor for Susan Baker called.

Employee constructor for Robert Jones called.

Number of employees after instantiation is 2

Employee 1: Susan Baker

Employee 2: Robert Jones

~Employee() called for Susan Baker

~Employee() called for Robert Jones

Number of employees after deletion is 0

THIS POINTER

- `this` is a pointer that allows every object to access its own address
- An object's `this` pointer is not part of that object i.e. `this` pointer is not reflected in the result of `sizeof()` operation on the object
- `this` pointer finds the address of the object dynamically (at run time) when an object is created
- `this` is passed into the object by the compiler as an implicit first argument on every non-static member function call to the object
- It is implicitly used to reference both data members and member functions of an object
- It can also be used explicitly
- The type of `this` pointer depends on the **type of the object** and whether the member function in which `this` is used **is declared const**

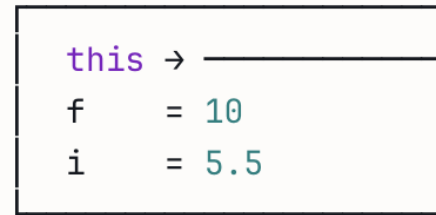
// What YOU write:

```
void Distance::setData(int f, float i)
{
    this->feet = f;
    this->inches = i;
}
```

// What COMPILER actually does behind the scenes:

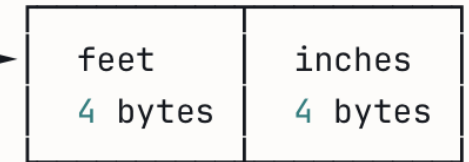
```
void Distance::setData(Distance* this, int f, float i)
{
    this->feet = f; // uses hidden pointer
    this->inches = i;
} ...
```

CALL STACK during setData():



(temporary – exists **only**
during function call)

HEAP/STACK (object memory):



(permanent – exists as
long as object lives)

WHAT IS AND IS NOT COUNTED IN SIZEOF()

```
class Example {
private:
    int    a;           // ✓ counted – 4 bytes
    float  b;           // ✓ counted – 4 bytes
    double c;           // ✓ counted – 8 bytes

public:
    static int count;    // ✗ NOT counted – belongs to class, not object

    Example() { }        // ✗ NOT counted – constructor
    void show() { }      // ✗ NOT counted – member function
                        // ✗ this pointer – NOT counted
};

int main() {
    Example e;
    cout << sizeof(e);   // 16 bytes (only a + b + c)
}
...
```

THIS POINTER

- In const member function `this` is used as a constant pointer to const object
 - In **const** member function of **Employee**
 - **this** has type **const Employee * const**
 - Constant pointer to constant **Employee** object
- In non-const member function `this` is used as a const pointer to non-const object
 - In non-**const** member function of **Employee**
 - **this** has type **Employee * const**
 - Constant pointer to non-constant **Employee** object

THIS POINTER AND STATIC MEMBERS

- A class may contains two types of members, static and non-static
- Static members of a class belong to **only that class and do not belong to objects**
 - Static members of a class are defined only once and not defined for every object
 - All objects share static members of the class
- **Non-static members belong to objects**
 - Non-static members are defined for every object
- **this belongs to object and does not belongs to class** and hence **it can not be used with static member functions**
- **this** is available in every non-static member function
- The compiler automatically/implicitly passes **this** pointer to all non-static member functions of a class

THIS POINTER AND STATIC MEMBERS

Cascade Member function calls

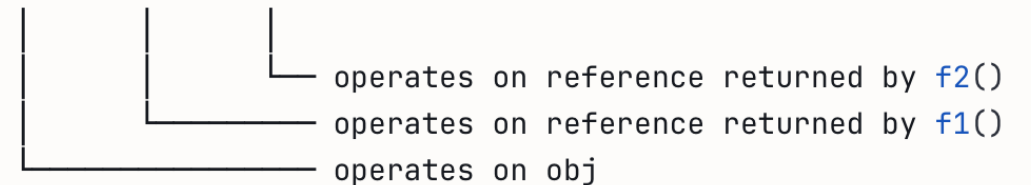
- We can invoke multiple member functions in same statement as,

```
object.function1().function2().function3().function4();
```

- The functions must return reference pointer to same object i.e.

```
return *this;
```

obj.f1().f2().f3()



- Other must operate on that pointer
- The functions that do not return reference must be called last

- ✓ Functions that RETURN reference (*this) → can be chained (called in middle)
- ✗ Functions that DON'T return reference → must be called LAST

```
1 // time6.h
2 // Cascading member function calls.
3
4 // Time class definition.
5 // Member functions defined in time6.cpp.
6 #ifndef TIME6_H
7 #define TIME6_H
8
9 class Time {
10
11 public:
12     Time( int = 0, int = 0, int = 0 ); // default constructor
13
14     // set functions
15     Time &setTime( int, int, int ); // set hour, minute, second
16     Time &setHour( int ); // set hour
17     Time &setMinute( int ); // set minute
18     Time &setSecond( int ); // set second
19
20     // get functions (normally declared const)
21     int getHour() const; // return hour
22     int getMinute() const; // return minute
23     int getSecond() const; // return second
24
```

Set functions return
reference to **Time** object
to enable cascaded
member function calls.

```
25    // print functions (normally declared const)
26    void printUniversal() const; // print universal time
27    void printStandard() const; // print standard time
28
29    private:
30        int hour;    // 0 - 23 (24-hour clock format)
31        int minute;  // 0 - 59
32        int second;  // 0 - 59
33
34    }; // end class Time
35
36    #endif
```

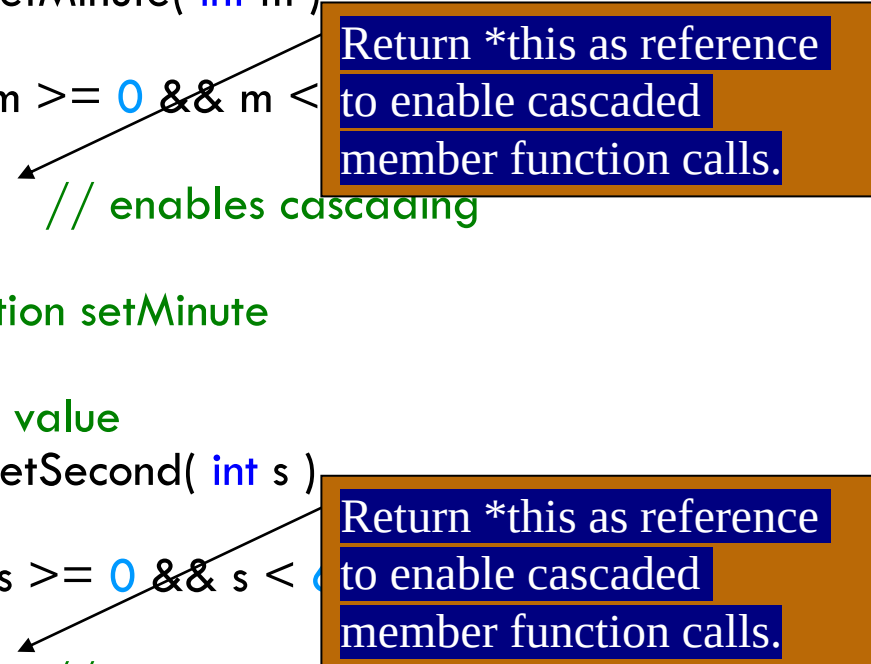
```
1  // time6.cpp
2  // Member-function definitions for Time class.
3  #include <iostream>
4
5  using std::cout;
6
7  #include <iomanip>
8
9  using std::setfill;
10 using std::setw;
11
12 #include "time6.h" // Time class definition
13
14 // constructor function to initialize private data;
15 // calls member function setTime to set variables;
16 // default values are 0 (see class definition)
17 Time::Time( int hr, int min, int sec )
18 {
19     setTime( hr, min, sec );
20
21 } // end Time constructor
22
```

```
23 // set values of hour, minute, and second
24 Time &Time::setTime( int h, int m, int s )
25 {
26     setHour( h );
27     setMinute( m );
28     setSecond( s );
29
30     return *this; // enables cascading
31
32 } // end function setTime
33
34 // set hour value
35 Time &Time::setHour( int h )
36 {
37     hour = ( h >= 0 && h < 24 ) ? h : 0;
38
39     return *this; // enables cascading
40
41 } // end function setHour
42
```

Return *this as reference
to enable cascaded
member function calls.

Return *this as reference
to enable cascaded
member function calls.

```
43 // set minute value
44 Time &Time::setMinute( int m )
45 {
46     minute = ( m >= 0 && m < 60 ) ? m : 0;
47
48     return *this; // enables cascading
49
50 } // end function setMinute
51
52 // set second value
53 Time &Time::setSecond( int s )
54 {
55     second = ( s >= 0 && s < 60 ) ? s : 0;
56
57     return *this; // enables cascading
58
59 } // end function setSecond
60
61 // get hour value
62 int Time::getHour() const
63 {
64     return hour;
65
66 } // end function getHour
67
```

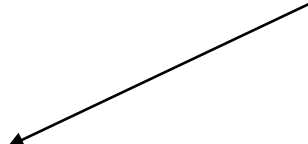


The diagram illustrates the concept of call-by-reference for cascading member function calls. It features two call boxes, one for `setMinute` (lines 44-49) and one for `setSecond` (lines 53-59). Both boxes contain the text: "Return *this as reference to enable cascaded member function calls." An arrow points from the `*this` in the `return` statement of `setMinute` (line 48) to the `Time &` part of the `setSecond` function signature (line 53). Another arrow points from the `*this` in the `return` statement of `setSecond` (line 57) to the `Time &` part of the `getHour` function signature (line 62). This visualizes how the reference to the object is passed along, allowing multiple member functions to be chained together.

```
68 // get minute value
69 int Time::getMinute() const
70 {
71     return minute;
72 }
73 // end function getMinute
74
75 // get second value
76 int Time::getSecond() const
77 {
78     return second;
79 }
80 // end function getSecond
81
82 // print Time in universal format
83 void Time::printUniversal() const
84 {
85     cout << setfill( '0' ) << setw( 2 ) << hour << ":"
86         << setw( 2 ) << minute << ":"
87         << setw( 2 ) << second;
88 }
89 // end function printUniversal
90
```

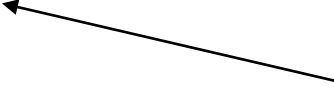
```
91 // print Time in standard format
92 void Time::printStandard() const
93 {
94     cout << ( ( hour == 0 || hour == 12 ) ? 12 : hour % 12 )
95         << ":" << setfill( '0' ) << setw( 2 ) << minute
96         << ":" << setw( 2 ) << second
97         << ( hour < 12 ? " AM" : " PM" );
98
99 } // end function printStandard
```

```
1  // mainprog.cpp
2  // Cascading member function calls with the this pointer.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  #include "time6.h" // Time class definition
9
10 int main()
11 {
12     Time t;
13
14     // cascaded function calls
15     t.setHour( 18 ).setMinute( 30 ).setSecond( 22 );
16
17     // output time in universal and standard formats
18     cout << "Universal time: ";
19     t.printUniversal();
20
21     cout << "\nStandard time: ";
22     t.printStandard();
23
24     cout << "\n\nNew standard time: ";
25
```



Cascade member function calls; recall dot operator associates from left to right.

```
26  // cascaded function calls
27  t.setTime( 20, 20, 20 ).printStandard();
28
29  cout << endl;
30
31  return 0;
32
33  } // end main
```



Function call to **printStandard** must appear last; **printStandard** does not return reference to **t**.

```
Universal time: 18:30:22
Standard time: 6:30:22 PM

New standard time: 8:20:20 PM
```

```

1  // prog.cpp
2  // Using the this pointer to refer to object members.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  class Test {
9
10 public:
11     Test( int = 0 );    // default constructor
12     void print() const;
13
14 private:
15     int x;
16
17 }; // end class Test
18
19 // constructor
20 Test::Test( int value )
21     : x( value ) // initialize x to value
22 {
23     // empty body
24
25 } // end Test constructor

```

ANOTHER EXAMPLE — THIS POINTER

```

26
27 // print x using implicit and explicit this pointers;
28 // parentheses around *this required
29 void Test::print() const
30 {
31     // implicitly use this pointer to access member
32     cout << "      x = " << x;
33
34     // explicitly use this pointer to access member x
35     cout << "\n this->x = " << this->x;
36
37     // explicitly use dereferenced this pointer and
38     // the dot operator to access member x
39     cout << "\n(*this).x = " << ( *this ).x << endl;
40
41 } // end function print
42
43 int main()
44 {
45     Test testObject( 12 );
46
47     testObject.print();
48
49     return 0;
50 } // end main

```

Implicitly use **this** pointer;
only specify name of data
member (**x**).

Explicitly use **this** pointer
with arrow operator.

Explicitly use **this** pointer;
dereference **this** pointer
first, then use dot operator.

```

      x = 12
    this->x = 12
  (*this).x = 12

```